To solve the first subtask, since $n \leq 10$, it will be possible to try all permutations of connection cutting and determine each permutation price using dfs or a disjoint set union structure.

The main caveat to solving other subtasks is that it will always be optimal to cut the connections around the maximum node in the tree first. Proof sketch: on the path between the hardest node of value M and the second hardest node of value m, we have to cut some connection at some point, and pay $m + M$. If the connection is closer to M, the cut is more cost effective because more nodes will be in the subtree whose maximum is $m \leq M$.

So, there will always be a solution that finds the maximum in the tree, cuts all the connections around it, finds maxima in newly formed trees, and recursively descends into them. It only remains to implement finding the maximum efficiently.

The second subtask, in which the graph is a chain, can be efficiently solved using the segment tree structure, which allows you to quickly search for the maximum at an interval (or subtree) with time complexity $O(n \log n)$.

In the third subtask ($n \leq 1000$) it is enough to search for the maximum using dfs, with time complexity $O(n^2)$. Now we solve the main subtask.

Claim: The solution is equal to

$$\sum_{i=1}^{n} t_i - \max_{1 \leq i \leq n} t_i + \sum_{i=1}^{n-1} \max(t_{x_i}, t_{y_i}).$$

Proof: Induct on n. Base case $n = 1$ is clear. When we cut a connection a—b_j around the maximum vertex a, we add $t_a + t_{a_j}$, where a_j is the maximum vertex in the subtree of b_j. Now adding up the formulas for the subtrees finishes the induction step.